

HelixAgent HelixLLM provider — LAN/VPN provider-alias proof

Revision: 2 **Last modified:** 2026-07-07 **Maintainer:** CLI-Agent Fusion programme (T1 / feature/helixllm-full-extension) **Scope:** Prove the HelixAgent helixllm.Provider works as an OpenAI-compatible provider alias over the LAN/VPN (not just localhost), env-var parameterized, with API-key auth — all with real captured evidence (§11.4 / §11.4.5 / §11.4.69 / §11.4.108 / §11.4.98).

Verdict: DONE

Every part (A env-var parameterization, B auth 401/200, C live LAN, D docs, E tests, F deep research) is landed with captured runtime evidence. No metadata- only PASS. The live coder (helixllm-coder, 0.0.0.0:18434) was never restarted/reconfigured (§11.4.122); the auth server was a fresh ephemeral process on :18439, torn down after use (§11.4.119).

Resume re-verification (post-crash, §11.4.147 respawn — 2026-07-07T16:25–16:27Z)

The prior attempt crashed on a shared session-limit right before commit. On resume, every unfakeable piece was RE-RUN on the CURRENT working tree (§11.4.6 — stale evidence is not trusted) and re-confirmed GREEN; fresh evidence:

Re-verify	File	Result
Unit + integration full suite (34 tests) on current tree + go vet	40_reverify_unit_integration.txt	ok ... 0.007s, EXIT=0, 0 failures
Raw LAN curl → real coder	41_reverify_raw_lan_curl.txt	HTTP 200 remote=10.6.100.221:18434

Re-verify	File	Result
Live LAN provider.Complete() via HELIX_LLM_HOST/ PORT	42_reverify_live_lan_provider.txt	content='LANPROOF-OK', Qwen3-Coder-30B PASS — endpoint http:// 10.6.100.221:18434, real func Add(a int, b int) int, 53 tok
Auth 401/200 matrix (ephemeral :18439) + real keyed provider	43_reverify_auth_matrix.txt	B1 401, B2 401, B3 200+func Add, B4 real provider 200+func Add

The build + all tests pass with the CURRENT go.mod (the foreign version-bump is NOT needed — see the §11.4.174 ownership note below).

Live-LAN runtime signature (§11.4.108 — the unfakeable core)

```
HELIX_LLM_HOST=10.6.100.221 HELIX_LLM_PORT=18434
→ provider.Endpoint() = "http://10.6.100.221:18434"    (composed,
no /v1)
→ provider.Complete() → HTTP 200, remote=10.6.100.221:18434
→ REAL coder output (Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf,
tokens_used=53):
    func Add(a int, b int) int { return a + b }
```

Raw wire cross-check (01_raw_lan_curl.txt): curl POST http://
10.6.100.221:18434/v1/chat/completions → HTTP 200
remote=10.6.100.221:18434, content:"LANPROOF-OK". The
remote=10.6.100.221:18434 (curl %{remote_ip}) proves the request
traversed the **LAN interface (eno1)**, not the loopback.

A. Env-var resolution (unit-proven — 10_unit_integration_full.txt)

New client-target vars on helixllm.Provider: HELIX_LLM_HOST (default localhost), HELIX_LLM_PORT (default 18434), HELIX_LLM_API_KEY (optional). Precedence (first non-empty wins), each run through normalizeBase:

#	Source	Example input	Resolved base	Test
1	explicit cfg.Endpoint	https://custom:9443	https://custom:9443	TestResolveEndpoint
2	HELIX_LLM_LOCAL_OPENAI_ENDPOINT	http://seam:8080	http://seam:8080	...ExplicitEndpoint
3	HELIX_LLM_ENDPOINT	https://gen:8443	https://gen:8443	...ExplicitEndpoint
4a	HELIX_LLM_HOST (LAN, port default)	10.6.100.221	http://10.6.100.221:18434	TestResolveEndpoint
4b	HELIX_LLM_PORT only (host default)	18434	http://localhost:18434	TestResolveEndpoint
4c	bind-all host mapped to localhost	0.0.0.0 / :: / [::]	http://localhost:18434	TestResolveEndpoint
5	nothing set (legacy)	—	https://localhost:8443	TestResolveEndpoint

No-double-/v1 invariant (TestNormalizeBase_NoDoubleV1, TestResolveEndpoint_NoDoubleV1Invariant): a base written either as http://h:18434 OR http://h:18434/v1 both compose to a single http://h:18434/v1/chat/completions — never /v1/v1 (the 404 gotcha).

B. Auth 401 / 200 matrix (20_auth_matrix_ephemeral.txt, 21_provider_keyed_lan.txt)

Ephemeral OpenAI server with a Bearer key on 0.0.0.0:18439 (the llama.cpp --api-key contract — byte-for-byte Bearer compare):

Case	Request	HTTP	Evidence
B1	no Authorization header	401	20_auth_matrix_ephemeral.txt
B2	Authorization: Bearer WRONG-KEY	401	20_auth_matrix_ephemeral.txt
B3	Authorization: Bearer <correct>	200 + real completion	20_auth_matrix_ephemeral.txt
B4	real provider HELIX_LLM_HOST=10.6.100.221 HELIX_LLM_PORT=18439	200 + func Add...	21_provider_keyed_lan.txt

Case	Request	HTTP	Evidence
	HELIX_LLM_API_KEY=<correct> → keyed server over LAN		

Provider-side auth is additionally proven end-to-end (no-key → 401, wrong-key → 401, correct-key → 200, gotAuth == "Bearer <key>") by the integration test `TestIntegration_HelixLLM_AuthMatrix` against a real `httptest` server.

C. Live LAN proofs

Proof	Command driver	Result	Evidence
C1	HELIX_LLM_HOST=10.6.100.221 (new composition) → <code>Complete()</code>	real func Add, 53 tok	30_live
C2	HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://10.6.100.221:18434 (base seam) → <code>Complete()</code>	real func Add, 53 tok	31_live
C3 (RED)	RED_MODE=1, no pin → TLS :8443 default	connection refused (defect reproduced §11.4.115)	32_reco
raw	<code>curl http://10.6.100.221:18434/...</code>	HTTP 200 remote=10.6.100.221:18434	01_raw

E. Tests (all GREEN)

- Unit + integration full suite: `10_unit_integration_full.txt` — ok ... 0.007s, 0 failures, existing suite unregressed.
- Re-runability at -count=3 (§11.4.98): `12_rerunnability_count3.txt` — ok.
- Files (helix_agent submodule, `internal/llm/providers/helixllm/`):
`provider.go` (env vars + `normalizeBase` + `composition`),
`provider_network_test.go` (unit resolution),
`provider_network_integration_test.go` (real-HTTP auth matrix + no-double-/v1 + HOST/PORT wiring), `provider_e2e_hostport_test.go` (live LAN), `provider_e2e_live_test.go` (reconciled assertion §11.4.120).

Reproduce

```
cd submodules/helix_agent
# unit + integration (no infra, deterministic):
go test -count=1 -v ./internal/llm/providers/helixllm/
# live LAN via HOST/PORT composition (coder must listen on
  0.0.0.0:18434):
HELIX_LLM_HOST=10.6.100.221 HELIX_LLM_PORT=18434 \
  go test -tags=helixllm_e2e -run
    TestE2E_HelixAgent_To_LiveHelixLLM_ViaHostPort -v \
    ./internal/llm/providers/helixllm/
# auth matrix: run an OpenAI server with a key on :18439, then
#   curl -o/dev/null -w '%{http_code}\n' -XPOST http://
      <HOST>:18439/v1/chat/completions          # 401
#   curl ... -H 'Authorization: Bearer WRONG'
      ...                                     # 401
#   curl ... -H 'Authorization: Bearer <correct>'
      ...                                     # 200
```

D. Docs

- submodules/helix_llm/docs/OPERATOR_GUIDE.md §7 — new **HelixAgent over the LAN / VPN** subsection (env-var table, precedence, host-IP examples, the /v1 base gotcha, API-key auth, api_keys.sh/.env), .html+.pdf siblings synced (§11.4.75).
- submodules/helix_agent/.env.example — client-connection block (precedence + the HELIX_LLM_HOST name-collision caveat with the server-bind block).
- submodules/helix_agent/api_keys.sh.example + .gitignore guard (api_keys.sh ignored, .example tracked — CONST-042/053/§11.4.10).

F. Deep research (§11.4.150 — 2 angles)

- Official llama.cpp server docs — --api-key (Bearer auth, HTTP 401 on missing/wrong), --host default 127.0.0.1, 0.0.0.0 binds all interfaces (LAN-reachable), --port default 8080: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md> (verified 2026-07-07).
- Community/issue corroboration — Authorization: Bearer <key> byte-for-byte compare → 401 on any mismatch (extra space/newline included): <https://markaicode.com/errors/llamacpp-api-key-invalid-fix-production/> (verified 2026-07-07); <https://github.com/ggml-org/llama.cpp/issues/22474> (verified 2026-07-07).

Honesty notes (§11.4.6)

- The :18434 coder fleet (raw llama-server) does **not** enforce auth — any/no key is accepted. The 401/200 auth matrix was therefore proven against (a) an ephemeral key-checking server on :18439 and (b) a real httptest server in the integration test; both mirror the llama.cpp --api-key Bearer contract. The provider's own header emission (Authorization: Bearer <key>) is proven real.
- The ephemeral auth server + the httptest servers are OpenAI-endpoint stand-ins, not provider mocks — the helixllm.Provider code path is 100% real (CONST-050). The live coder proofs (C1/C2, 01_raw_lan_curl.txt) use the real model.
- HELIX_LLM_HOST/HELIX_LLM_PORT names are reused from the existing .env.example “Server Settings” (server bind). The client composition sits BELOW the endpoint seams in precedence, so a default deployment (HELIX_LLM_ENDPOINT set) is unaffected; a leaked bind-all 0.0.0.0 maps to localhost. Documented explicitly in .env.example.
- Not verified here: the full :8443 HelixLLM Go gateway live-serving (its contract is documented in API_CONTRACT.md, not exercised in this session).
- **§11.4.174 ownership honesty note (foreign working-tree files).**
The submodules/helix_agent working tree also carries CONCURRENT-track changes that are **NOT part of this feature and were NOT staged/committed/reverted by this work**: go.mod + go.sum (an 18-line go.opentelemetry.io/otel + golang.org/x/* version bump) and their go.mod.qa_bak + go.sum.qa_bak backups. This provider work adds **NO new external import** (provider.go uses only stdlib net/http/os/strings/... + the existing dev.helix.agent/internal/{llm,models}); the tests use net/http/httptest + testify), so it **builds and all 34 tests pass against the CURRENT unmodified go.mod** — proven by 40_reverify_unit_integration.txt (go vet clean, ok ... EXIT=0). Those four files were left untouched; only the eight files owned by this feature were committed.

Sources verified 2026-07-07

- <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- <https://markaicode.com/errors/llamacpp-api-key-invalid-fix-production/>
- <https://github.com/ggml-org/llama.cpp/issues/22474>